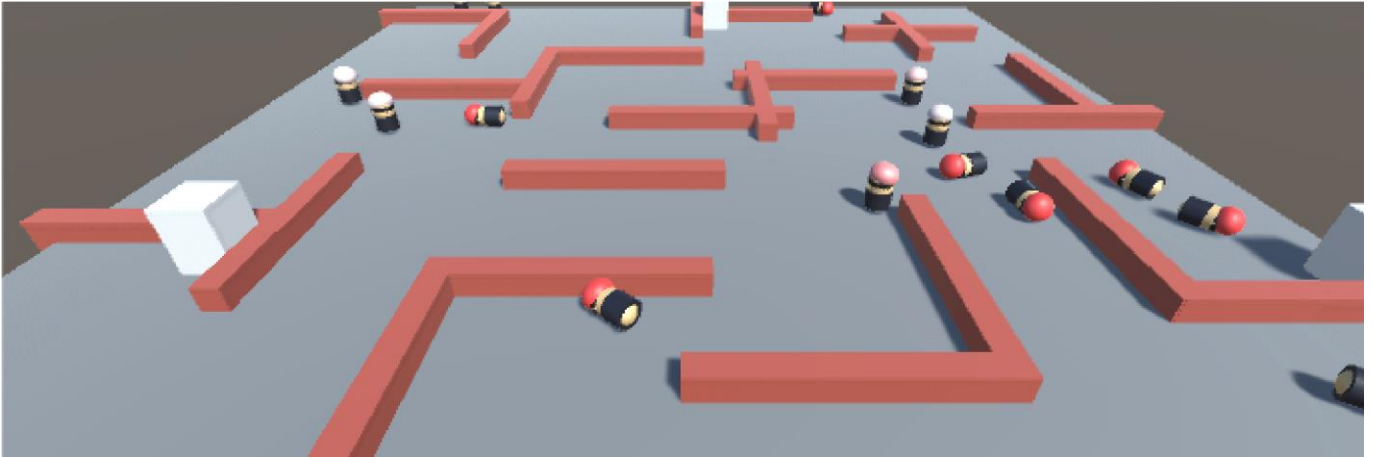


Plague in the City

CU6051 Artificial Intelligence

Disease Propagation Simulation



Student Name	Samuel Akintomide	Student Name	Erza Mripa
ID	22041893	ID	21051876
Supervisor	Fiona French		
Academic Year	2023-24		

Table of Contents

1.0 Overview.....	3
2.0 Introduction	3
2.1 Viruses and Pandemics in Modern Society	3
2.2 Applying ‘Plague in the City’ in Everyday Situations.....	3
2.3 Enhancing Navigation Through Environmental Data Collection	4
3.0 Design	4
3.1 PEAS for Problem/Space Environment	4
3.2 Design Pattern/Architecture/UML Diagrams	4
3.2.1 Design Patterns.....	4
3.2.2 Architecture Diagram	4
3.2.3 UML Diagram	5
3.3 Creating ‘Plague in the City’	6
3.4.0 Code.....	9
3.5 Challenges	10
3.6 Amendments to the Project.....	10
3.7 Added Features to the Simulation.....	10
4.0 Conclusion	13
5.0 Appendix A – Code	14
A1 – Scene Spawner (.cs)	14
A2 – Agent Movement (.cs)	16
A3 – Movement (.txt)	17
A4 – Dead Agent (.cs)	17
A5 – Find Dead (.cs)	18
A6 – Find Dead (.txt)	19
A7 – Health Manager (.cs).....	19
A8 – Genes (.cs)	21
A9 – Medical Box (.cs)	22
A10 – Rotation (.cs).....	22
A11 – Main Menu (.cs)	22
A12 – Simulation Settings (.cs)	23
6.0 References	24

1.0 Overview

This project aims to leverage artificial intelligence to simulate the spread of various disease strains in different environments, particularly within a virtual city setting. The goal is to create a dynamic digital environment where virtual agents represent city residents, moving about, interacting, and potentially transmitting a virus.

Agents in the simulation exhibit realistic behaviours, with their health status influenced by contact with contaminants or interactions with infected peers. Without intervention, infected agents may continue spreading the disease. To mitigate the spread, the simulation deploys special agents equipped with simulated antiviral treatments. These agents actively move through identified hotspots, aiding in the faster recovery of infected individuals. Additionally, strategically positioned virtual treatment centres offer further support for infected agents. Advanced AI algorithms control these elements, facilitating a dynamic representation of a pandemic from its initial outbreak to containment. Researchers can experiment with variables such as agent health, virus characteristics, and urban planning strategies. This experimentation yields valuable insights into managing real-world infectious threats and supports informed public health decisions.

In summary, the project demonstrates how AI can simulate a complex city-scale pandemic scenario. By guiding virtual agents' behaviours, health dynamics, and strategic interventions, the simulation provides a powerful tool for analysing infectious disease dynamics and exploring effective containment strategies.

2.0 Introduction

This project explores the intersection of artificial intelligence and healthcare by simulating disease dynamics and testing the efficacy of treatments in a virtual environment. The simulation models how diseases spread, how agents respond to medical interventions, and how AI-driven strategies can contain outbreaks. The report details the creation of this AI-powered simulation, showcasing the design choices, challenges, and insights gained. By simulating realistic agent behaviours in a dynamic city-like environment, the project provides a platform for researchers to explore innovative approaches to disease management and containment strategies.

2.1 Viruses and Pandemics in Modern Society

Viruses and pandemics pose significant threats to global health, economies, and daily life. The rapid spread of infections, such as COVID-19, highlights the need for effective responses in an increasingly interconnected world. Addressing these challenges requires ongoing research and investment in healthcare systems to develop strategies that mitigate the impact of future outbreaks.

2.2 Applying 'Plague in the City' in Everyday Situations

This project offers valuable insights that can inform public health strategies and behavioural changes to better contain disease outbreaks. Potential applications include:

- **Public Health Campaigns:** Simulation results can inform campaigns about disease spread and promote preventative measures such as handwashing, mask-wearing, and social distancing.
- **Travel Guidance:** Health authorities can use the simulator to assess risk levels for specific locations and provide travel advisories.
- **School/Workplace Policy:** The simulator can demonstrate the impact of early closures and reopening during outbreaks, guiding decision-makers.
- **Individual Behaviour:** By visualising how personal actions affect disease transmission, the simulation can encourage people to adopt healthier practices, even when they feel at low risk.

2.3 Enhancing Navigation Through Environmental Data Collection

Integrating environmental data collection into route mapping has great potential for enhancing navigation efficiency. By using sensor technologies and datasets on terrain, obstacles, and landmarks, advanced algorithms can find optimal paths considering landscape variables. In the "Plague in the City" simulation, navigation meshes, and agent parameters improved dynamic movements and obstacle avoidance. Expanding on this, real-time data can help mapping systems offer personalised, environment aware routes. Absorbing comprehensive environmental data allows adaptation to changing landscapes. Combining rich data with smart mapping algorithms sets the stage for the next generation of agile, location-aware navigation services.

3.0 Design

3.1 PEAS for Problem/Space Environment

- **Performance:** Agent performance is measured by the duration they survive before being affected by the simulated disease. Health status and agent longevity provide insights into disease dynamics.
- **Environment:** The environment consists of a virtual city landscape with obstacles and structures resembling buildings. Hot zones with higher contagion rates present additional challenges.
- **Actuators:** Agents use autonomous movement capabilities through embedded pathfinding and steering algorithms, enabling behaviours such as wandering and exploring infected zones.
- **Sensors:** Agents are equipped with sensors for obstacle detection and health status awareness. These sensors help agents avoid collisions and react to nearby health threats by steering clear of contagious individuals when possible.

3.2 Design Pattern/Architecture/UML Diagrams

3.2.1 Design Patterns

- Observer pattern - Alert agents when agents in the vicinity become infected or pass away. Agents can now respond to state changes in other agents thanks to this.
- Strategy pattern - Utilised to implement various movement algorithms, such as pathfinding, flocking, and random wandering. This makes it simple to extend or modify the behaviour of the agent by putting new strategies into practice.
- State pattern - This pattern is used to control the agents' various health states (well, infected, and dead). This transitions in the state objects in a clear manner and captures state-specific behaviour.

3.2.2 Architecture Diagram

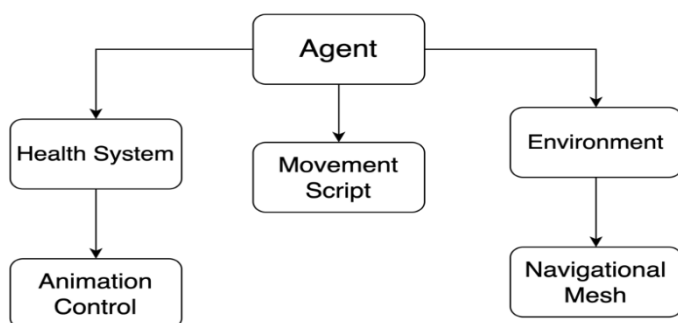


Figure 1: Architectural Diagram of the Simulation. (draw.io, 2021)

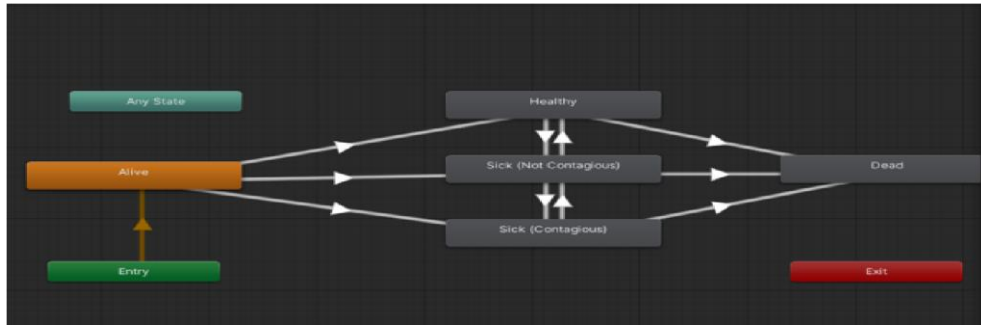


Figure 2: Architectural Diagram of the Simulation Generated Automatically by Unity. (draw.io, 2021)

3.2.3 UML Diagram

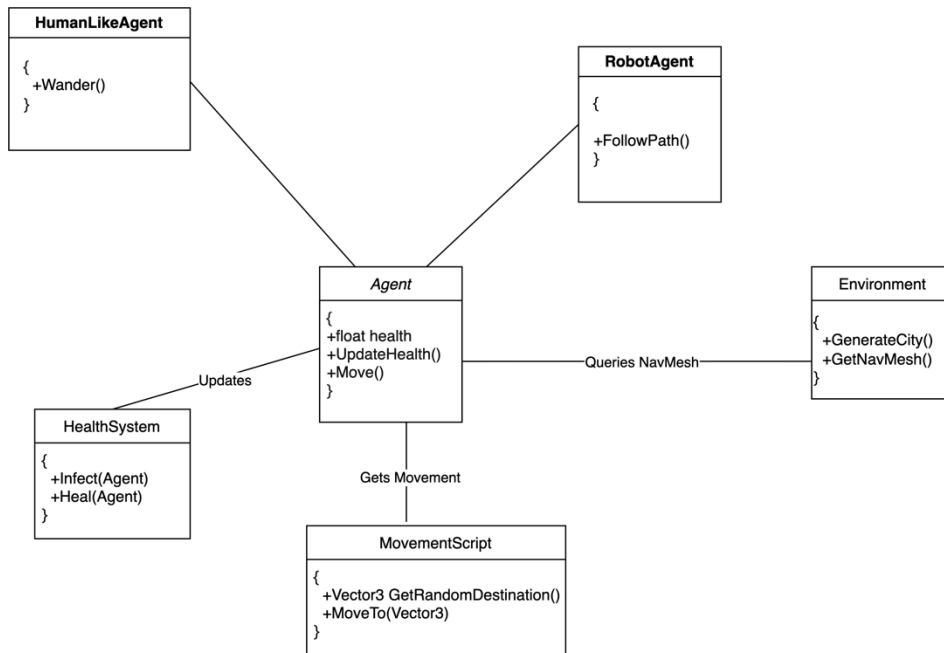


Figure 3: UML Diagram Demonstrating the Simulation. (draw.io, 2021)

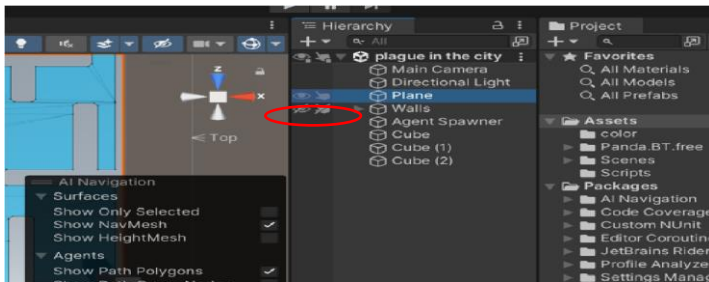
Key Classes and Relationships:

- Different agent types inheriting from base Agent class.
- Different methods on Agent and sub-classes
- Environment handles environment generation.
- MovementScript handles agent movement.

3.3 Creating 'Plague in the City'

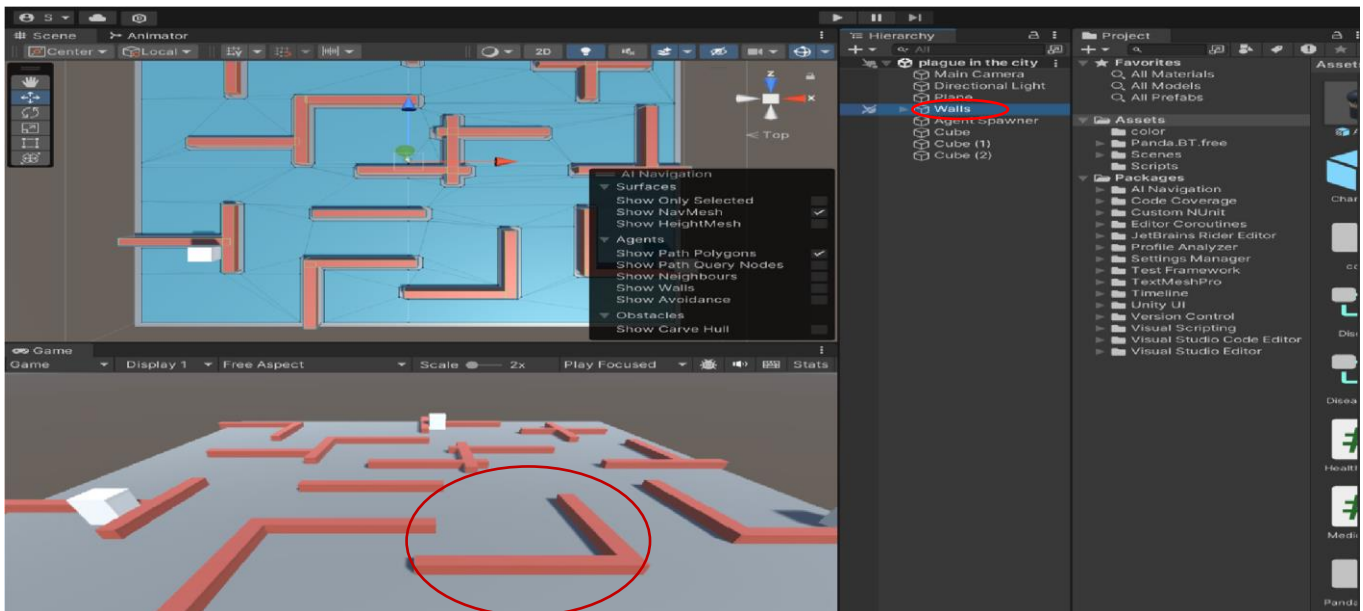
Step 1: Establishing the Base Environment

Created a foundational flat plane object to serve as the ground for agents to stand on.



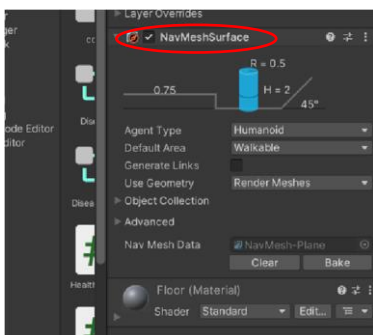
Step 2: Urbanisation through Obstacle Integration

Introduced obstacles resembling buildings to simulate a city environment, enhancing the complexity of agent navigation.



Step 3: Navigation Mesh Integration

Implemented a navigation mesh (nav-mesh) to enable agents to identify and navigate around obstacles, facilitating efficient pathfinding.



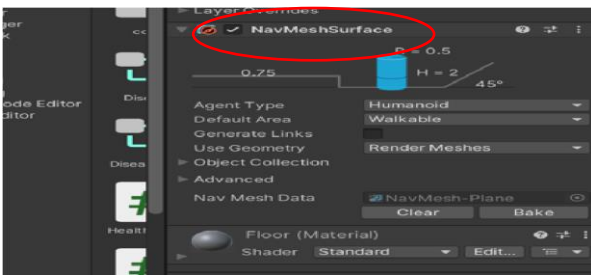
Step 4: Obstacle Interaction with Colliders

Added colliders to building objects to prevent agents from traversing through them, effectively creating physical barriers.



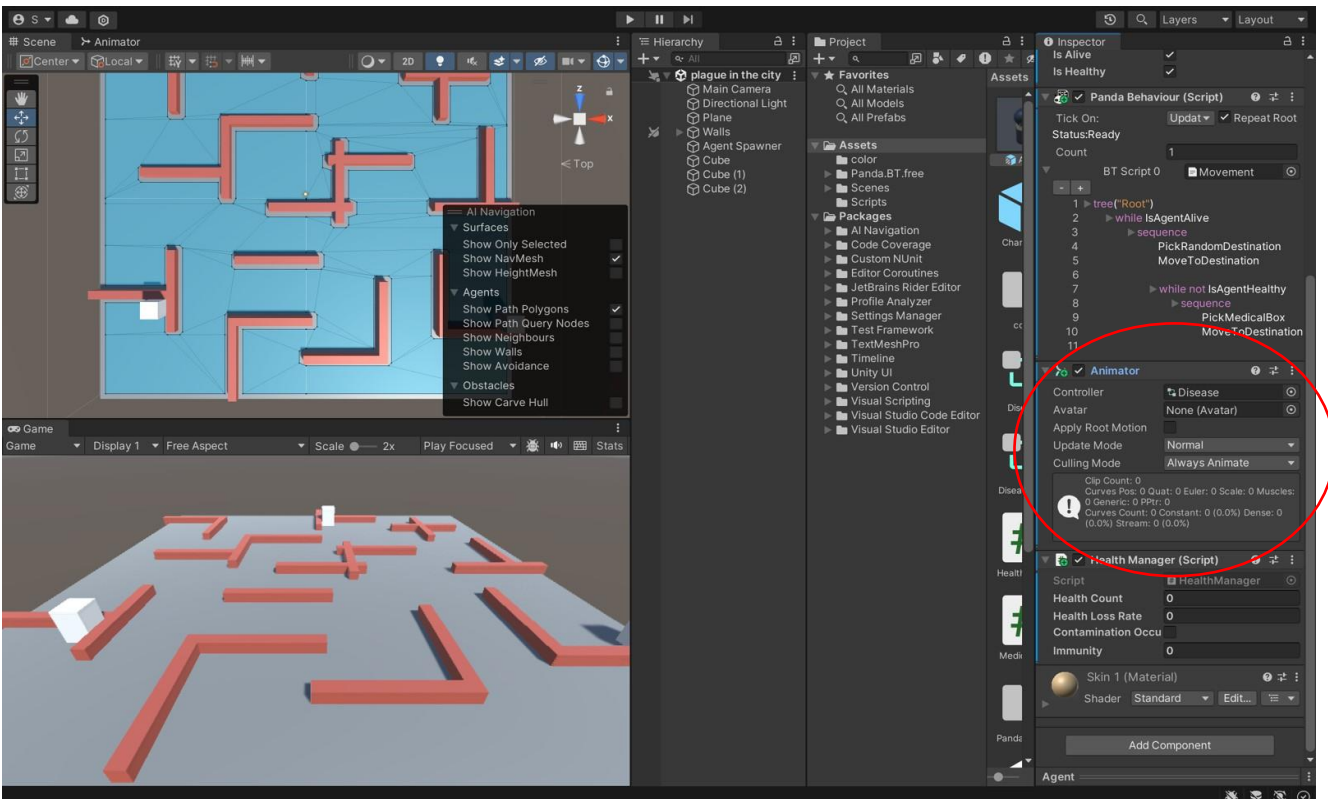
Step 5: Configuring Navigation Parameters

Integrated a navigation mesh agent window to modulate agent speed, acceleration, and facilitate obstacle avoidance during pathfinding.



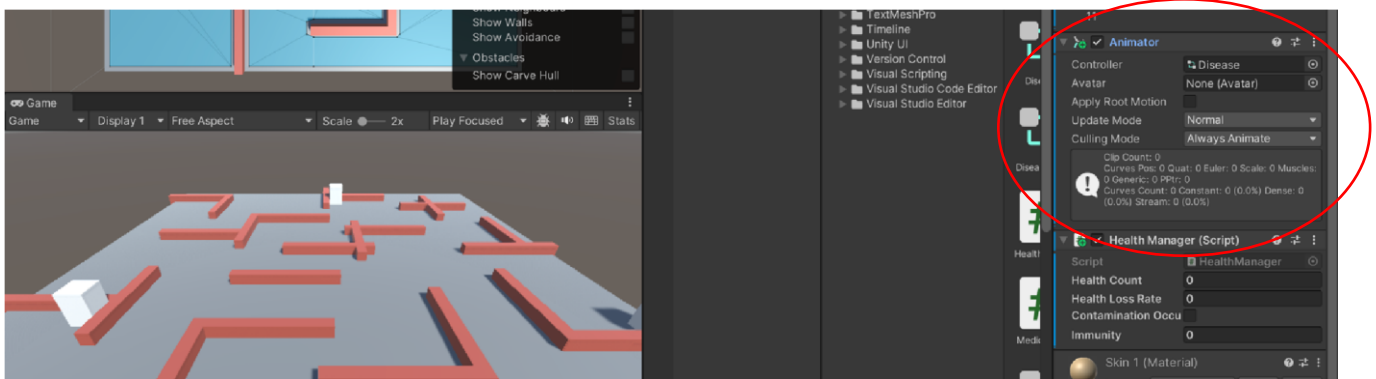
Step 6: Implementing Random Movement (Wandering) with Panda

Utilised the Panda framework to implement a wandering behaviour, allowing agents to move aimlessly throughout the city-like environment.



Step 7: Dynamic Animation Control

Implemented an animation controller that visually represents the health status of each agent, including indicators for infection, healthiness, contagion, and mortality.

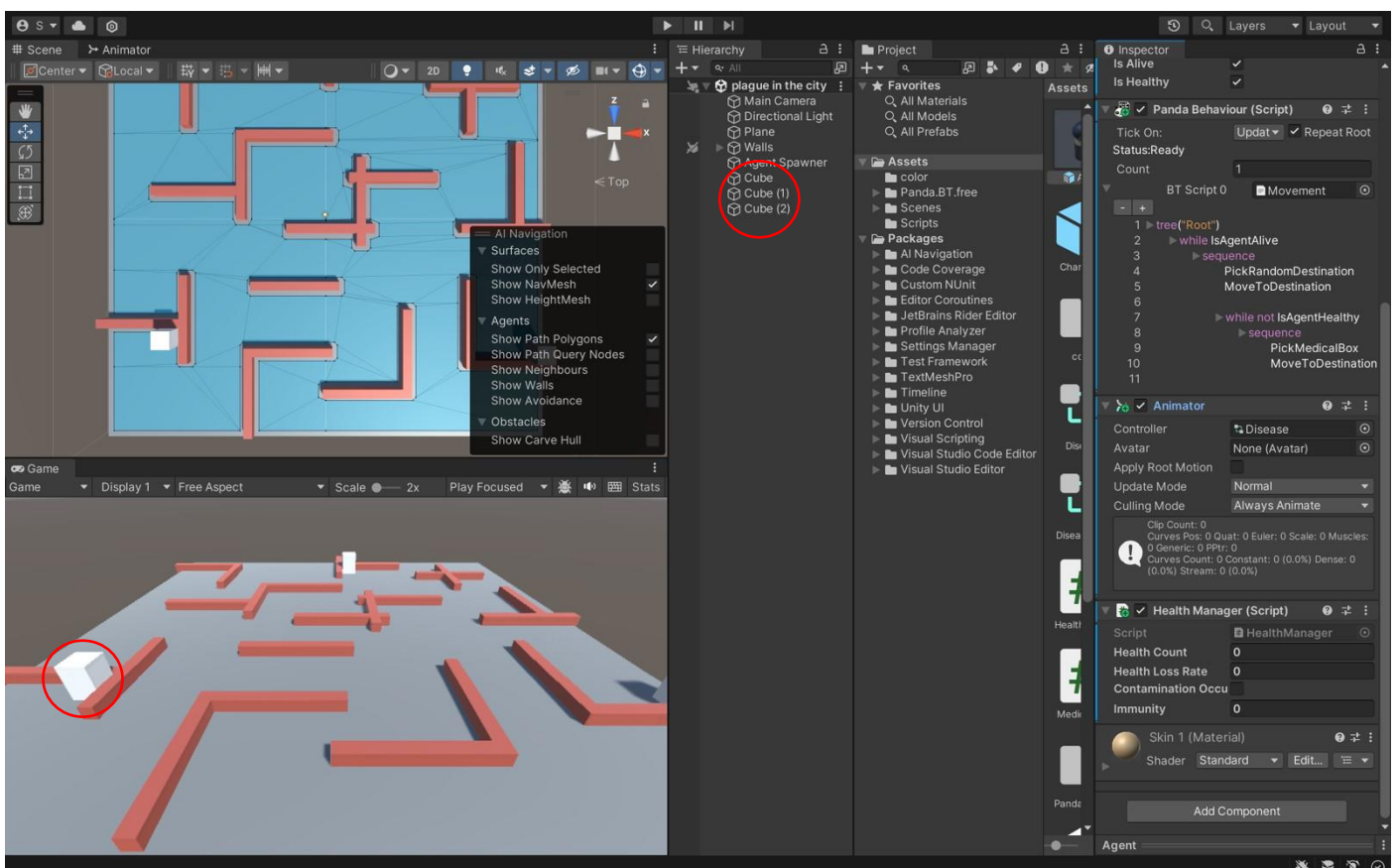


Step 8: Health Dynamics and Contagion Mechanism: Scripts

Programmed 'Infection' scripts to simulate the dynamic health changes of agents, accounting for individual health degradation and accelerated decline when in proximity to contagious agents.

Step 9: Implementing Healing Mechanics

Integrated a healing system where the close proximity of agents to designated big white boxes initiates a dynamic regeneration process, fostering the recovery of infected agents.



3.4 Code

All the code used in this project is detailed in Appendix A, along with an overview of each component and its contribution to the overall functionality of the programme. The project consists of multiple scripts and assets, each responsible for handling different aspects of the simulation, including agent behaviours, environment interactions, user interface, and scene management. By breaking down these components, we can better understand how they come together to form a cohesive system, enabling efficient and interactive simulation execution. The overview highlights key functions, control logic, and communication between scripts, providing insight into the design decisions and problem-solving approaches used throughout the project development.

The **Scene Spawner (.cs)** script plays a critical role in setting up the initial state of the simulation. It dynamically positions agents within the environment and integrates UI elements to monitor their health status. Through interactive features, users can select individual agents via mouse clicks to check their health and activate agent-specific cameras. Additionally, this script manages the count of living and dead agents, providing real-time feedback on the simulation's progress.

The **Agent Movement (.cs)** script governs the navigation and movement of agents using Unity's NavMeshAgent component. Virtual agents select random destinations when healthy or move toward available medical resources if unwell. This script also allows programmatic changes to an agent's health status and behaviour, enabling smooth transitions between movement states depending on simulation conditions. Complementing the movement logic, the **Movement (.txt)** behaviour tree outlines sequential tasks for agents, such as exploring the environment and seeking health resources when necessary. This hierarchical structure ensures agents respond dynamically to changing events within the simulation.

To manage agent death scenarios, the **Dead Agent (.cs)** script is triggered upon an agent's death. It removes the agent from the visible scene and makes it non-interactive, maintaining the simulation's performance by checking proximity to other agents and clearing inactive elements. The **Find Dead (.cs)** and **Find Dead (.txt)** components work together to enable agents to locate and approach dead agents. The behaviour tree defines task sequences that ensure agents prioritize and navigate toward the nearest deceased individuals, leveraging Unity's NavMeshAgent for precise movement.

Health management is a central aspect of the simulation, handled by the **Health Manager (.cs)** script. This component simulates dynamic health transitions as agents progress through various states: healthy, sick, contagious, and dead. It manages interactions between agents, simulating disease spread and recovery processes through contact with medical resources, thereby adding complexity and realism to the simulation. The **Genes (.cs)** script provides a visual representation of agent health by dynamically adjusting agent colors. Healthier agents appear whiter, while sicker agents take on a redder hue, offering users a quick visual indicator of the simulation's health dynamics.

Supporting health recovery, the **Medical Box (.cs)** acts as a placeholder for interactive health resources. When agents come into proximity, these components help restore agent health.

Environmental enhancements are handled by the **Rotation (.cs)** script, which allows objects to rotate at configurable speeds. This feature is often applied to visual indicators or environmental elements to improve user engagement.

The simulation's control and customization functionalities are managed by two key scripts. The **Main Menu (.cs)** script facilitates scene transitions between the main menu and the simulation environment, enabling users to start and stop the simulation easily. Meanwhile, the **Simulation Settings (.cs)** script captures user input for configuring various parameters, including agent count, disease severity, and immunity levels. These settings are saved using PlayerPrefs and dynamically applied during the simulation.

These core components work together to create a dynamic, interactive simulation environment that offers users insight into agent behaviours, health dynamics, and environmental interactions

3.5 Challenges

Regarding the challenges we faced when developing the simulation, the main issues stemmed from trying to develop features that would make the application more user-friendly. For example, when attempting to make it easy to identify the agents who were sick against those that were not, we decided that by using a large common object on our agents (being their hair), it would make it easier to differentiate the former from the latter. Connecting the health values to a different object on the agent caused some issues which we later were able to get around by making amendments to the script.

Additionally, pairing the settings menu to affect the values in the simulation initially caused us an issue but it didn't take too long before we found the solution. This issue arose because we weren't aware of the need for scene management when using scripts that carry data across two or more different scenes.

We also faced issues pertaining to the interactions with agents so that the user could see their information and the medical agents so that the dead bodies of other agents weren't just lying dormant on the floor, inadvertently acting as new obstacles for other agents trying to navigate the environment.

3.6 Amendments to the Project

An intriguing idea emerged during the development phase, to incorporate the project into ChatGPT, enabling agents to provide comments. This addition was seen as a potential enhancement, fostering a more interactive and dynamic simulation environment.

3.7 Added Features to the Simulation



Figure 4: A newly designed Settings Page for the Simulation.



Figure 5: A newly designed Menu Page for the Simulation.

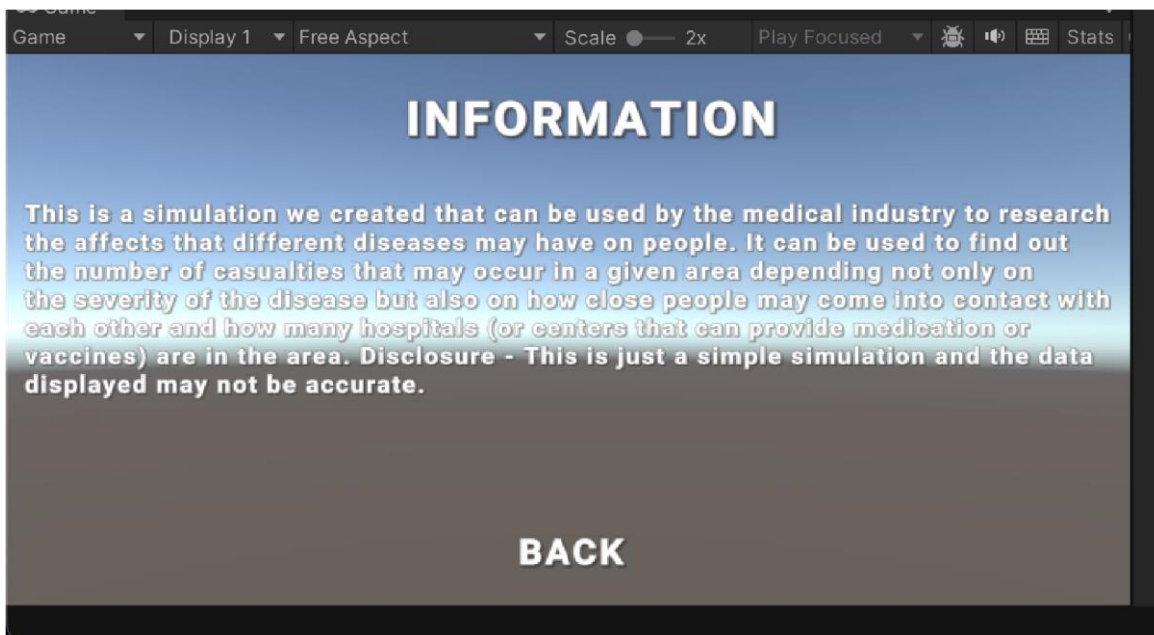


Figure 6: A newly designed Information's Page for the Simulation.

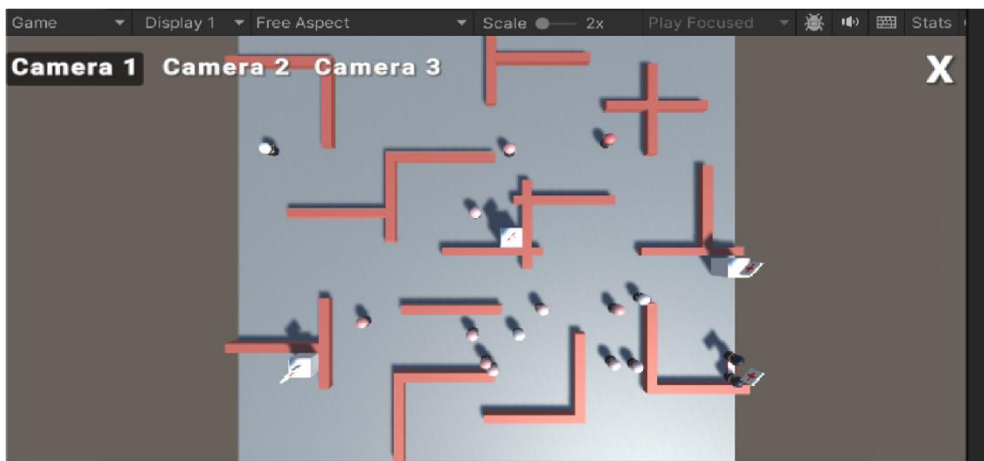


Figure 7: Showing the birds eye view of Camera 1, (one of the new cameras that were added).

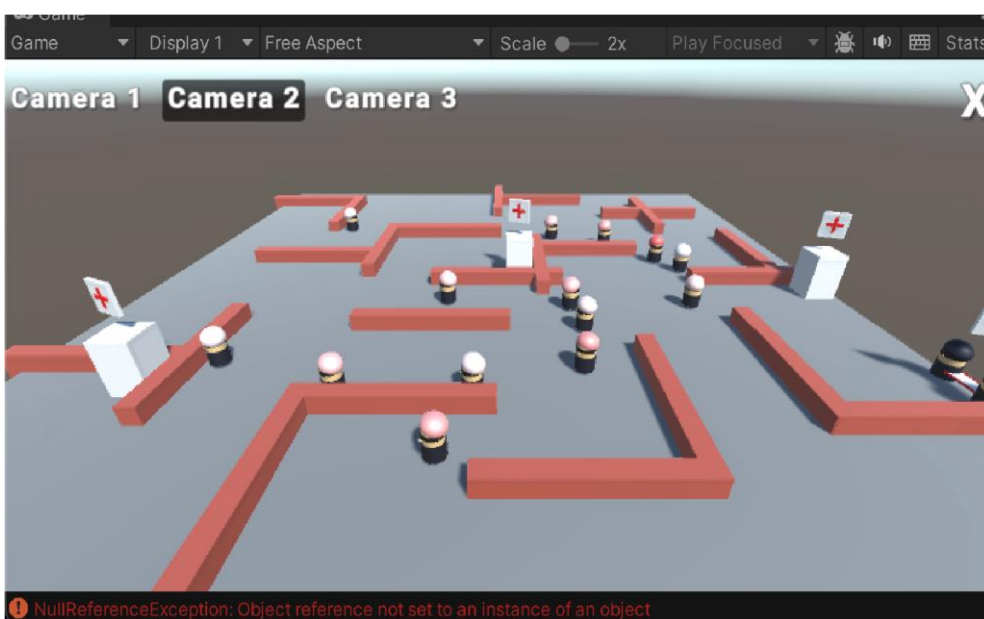


Figure 8: Showing the up high view of Camera 2, (the original camera).

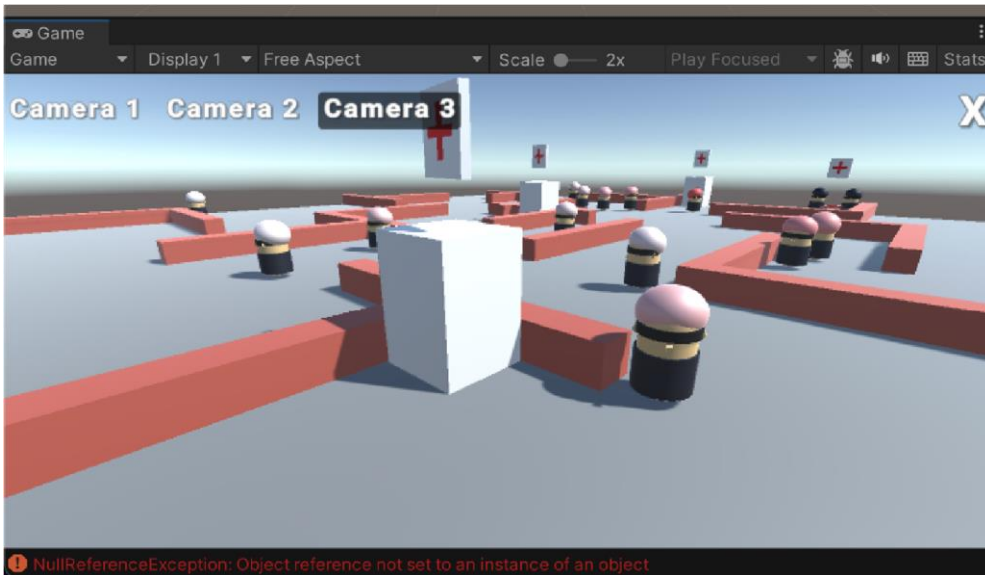


Figure 7: Showing the up-close view of Camera 3, (one of the new cameras that were added).

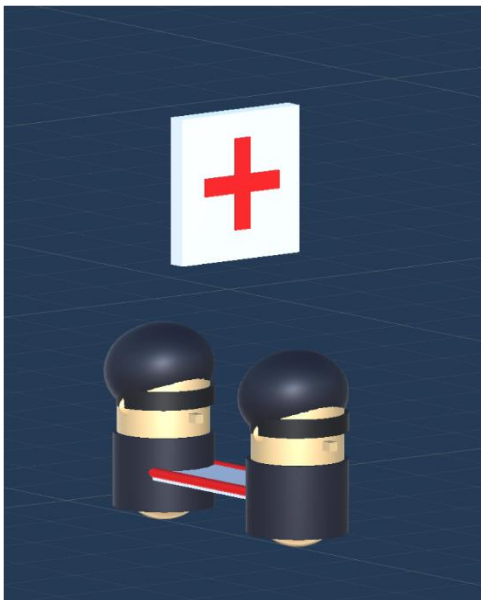


Figure 8: Showing the specially designated agents imitating Doctors.

An added feature to the 'Plague in the City' are these specially designated agents that conduct periodic rescue missions, transporting deceased agents to safezones for recovery. They possess navigational skills enabling them to traverse the cityscape efficiently like the other agents. These Agents equipped with health-sensing capabilities detect peers in distress, proactively navigating towards them to facilitate transport to the designated hospital.

4.0 Conclusion

In conclusion, the development of this AI simulation for disease spread and medication efficacy has been a journey marked by both challenges and innovative solutions. The step-by-step approach allowed for the creation of a dynamic and realistic virtual environment, providing valuable insights into the complexities of disease transmission and the impact of medical interventions.

The challenges faced, particularly in implementing the healing mechanism through medicine capsules and the strategic placement of these capsules, underscore the intricate nature of simulating real-world scenarios. These hurdles were overcome through persistent problem-solving, emphasising the importance of adaptability and creativity in the realm of AI simulations.

The consideration to integrate the project into ChatGPT, enabling agents to offer comments, presents an exciting avenue for enhancing user interaction and expanding the simulation's capabilities. This potential addition opens doors to a more immersive and engaging experience, fostering a deeper understanding of disease dynamics.

As we reflect on the steps taken from creating a realistic city-like environment to implementing navigation systems, health dynamics, and behavioural scripts the resulting simulation stands as a testament to the interdisciplinary nature of AI research. The fusion of programming, animation, and artificial intelligence techniques has yielded a comprehensive tool for studying and understanding the intricate interplay between diseases, environments, and medical interventions.

Ultimately, this AI simulation serves as a valuable resource for researchers, educators, and healthcare professionals, offering a controlled platform to observe, analyse, and experiment with various disease scenarios. The continual refinement and expansion of this simulation hold the promise of contributing to advancements in the field of healthcare and AI, furthering our understanding of disease dynamics and the efficacy of potential treatments.

5.0 Appendix A

A1 – Scene Spawner (.cs)

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class SceneSpawner : MonoBehaviour{
    public GameObject agentPrefabs;
    public GameObject[] allAgents;
    public GameObject uiPanel;
    public GameObject[] cameras;
    public TextMeshProUGUI healthText;
    public TextMeshProUGUI immunityText;
    public TextMeshProUGUI hospitalProximityText;
    public TextMeshProUGUI timeAliveText;
    public TextMeshProUGUI aliveCountText;
    public TextMeshProUGUI deadCountText;
    Vector3 startPos;
    public int numAgents;
    private GameObject lastClickedAgent;
    private bool isUpdatingHealth;
    private int aliveAgentCount;
    private int deadAgentCount;
    public static SceneSpawner Instance;

    void Awake(){
        Instance = this;
    }

    void Start(){
        numAgents = PlayerPrefs.GetInt("NumAgents");
        allAgents = new GameObject[numAgents];
        aliveAgentCount = numAgents;
        deadAgentCount = 0;
        for (int i = 0; i < numAgents; i++){
            float xPos = Random.Range(-30, 30f);
            float zPos = Random.Range(-30, 30f);
            startPos = new Vector3(xPos, 1f, zPos);
            allAgents[i] = Instantiate(agentPrefabs, startPos, Quaternion.identity);
            allAgents[i].GetComponentInChildren<Camera>().enabled = false;
        }
        UpdateAgentCountText();
    }

    void Update(){
        if (Input.GetMouseButtonDown(0)){
            RaycastHit hitInfo;
            Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);
            if (Physics.Raycast(ray, out hitInfo)){
```

```

GameObject clickedAgent = GetClickedAgent(hitInfo.collider.gameObject);
if (clickedAgent != null){
    uiPanel.SetActive(true);
    lastClickedAgent = clickedAgent;
    HealthManager agentHealthManager = clickedAgent.GetComponent<HealthManager>();
    if (agentHealthManager != null){
        StartCoroutine(UpdateHealthText(agentHealthManager));
    }

    // Enable camera if found
    Camera agentCamera = clickedAgent.GetComponentInChildren<Camera>();
    if (agentCamera != null){
        agentCamera.enabled = true;
    } else {
        Debug.LogWarning("Camera component not found in clicked agent's children.");
    }
} else {
    uiPanel.SetActive(false);
    if (lastClickedAgent != null){
        Camera lastCamera = lastClickedAgent.GetComponentInChildren<Camera>();
        if (lastCamera != null){
            lastCamera.enabled = false;
        }
    }
}
}
}
}
}
}

```

```

IEnumerator UpdateHealthText(HealthManager agentHealthManager){
    isUpdatingHealth = true;
    while (lastClickedAgent == agentHealthManager.gameObject){
        healthText.text = agentHealthManager.healthCount.ToString("F2");
        immunityText.text = agentHealthManager.immunity.ToString("F2");
        timeAliveText.text = agentHealthManager.time.ToString("F2");
        yield return null;
    }
    isUpdatingHealth = false;
}

```

```

GameObject GetClickedAgent(GameObject clickedObject){
    for (int i = 0; i < numAgents; i++){
        if (clickedObject == allAgents[i]){
            return allAgents[i];
        }
    }
    return null;
}

```

```

public void AgentDied(){
    deadAgentCount++;
    aliveAgentCount--;
    UpdateAgentCountText();
}

```



```

}

void UpdateAgentCountText(){
    aliveCountText.text = "" + aliveAgentCount;
    deadCountText.text = "" + deadAgentCount;
}
}

```

A2 – Agent Movement (.cs)

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.AI;
using Panda;

public class Robot_AI : MonoBehaviour
{
    NavMeshAgent agent;
    public bool isAlive = true;
    public bool isHealthy = true;

    void Start(){
        agent = this.GetComponent<NavMeshAgent>();
        agent.stoppingDistance = 5.0f;
    }

    public void SetAliveState(bool alive){
        isAlive = alive;
        if (!isAlive){
            agent.isStopped = true;
        }
    }

    public void SetHealthyState(bool healthy){
        isHealthy = healthy;
    }

    [Task]
    public bool IsAgentAlive(){
        return isAlive;
    }

    [Task]
    public bool IsAgentHealthy(){
        return isHealthy;
    }

    [Task]
    public void PickRandomDestination(){
        Vector3 dest = new Vector3(Random.Range(-30,30), 0, Random.Range(-30,30));
        agent.SetDestination(dest);
        Task.current.Succeed();
    }
}

```

[Task]

```
public void MoveToDestination(){
    if(Task.isInspected){
        Task.current.debugInfo = string.Format("t={0:0.00}", Time.time);
    }
    if(agent.remainingDistance <= agent.stoppingDistance && !agent.pathPending){
        Task.current.Succeed();
    }
}
```

[Task]

```
public void PickMedicalBox(){
    Collider[] colliders = Physics.OverlapSphere(transform.position, 30.0f);
    MedicalBox closestMedicalBox = null;
    float closestDistance = float.MaxValue;
    foreach (Collider collider in colliders){
        MedicalBox medicalBox = collider.GetComponent<MedicalBox>();
        if (medicalBox != null){
            float distance = Vector3.Distance(transform.position, medicalBox.transform.position);
            if (distance < closestDistance){
                closestMedicalBox = medicalBox;
                closestDistance = distance;
            }
        }
    }
    if (closestMedicalBox != null){
        Vector3 dest = new
Vector3(closestMedicalBox.transform.position.x,0,closestMedicalBox.transform.position.z);
        agent.SetDestination(dest);
        Task.current.Succeed();
    }
}
```

A3 – Movement (.txt)

```
tree("Root")
while IsAgentAlive
    sequence
        PickRandomDestination
        MoveToDestination

    while not IsAgentHealthy
        sequence
            PickMedicalBox
            MoveToDestination
```

A4 – Dead Agent (.cs)

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.AI;

public class DeadAgent : MonoBehaviour{
```

```

void Start(){
    enabled = false;
}

void Update(){
    if (enabled == true){
        FindDead[] allAgents = GameObject.FindObjectsOfType<FindDead>();
        foreach (FindDead agent in allAgents){
            float distance = Vector3.Distance(transform.position, agent.transform.position);
            if (distance < 1.5f){
                transform.position = new Vector3(transform.position.x, -200f, transform.position.z);
                Destroy(gameObject);
            }
        }
    }
}

public void EnableScript(){
    enabled = true;
}
}

```

A5 – Find Dead (.cs)

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.AI;
using Panda;

public class FindDead : MonoBehaviour{
    NavMeshAgent agent;
    public bool go = true;
    public bool isAlive = true;

    void Start(){
        agent = GetComponent<NavMeshAgent>();
        agent.stoppingDistance = 1.0f;
    }

    [Task]
    public bool IsAgentAlive(){
        return isAlive;
    }

    [Task]
    public void MoveToDestination(){
        if (Task.IsInspected){
            Task.current.debugInfo = string.Format("t={0:0.00}", Time.time);
        }
        if (agent.remainingDistance <= agent.stoppingDistance && !agent.pathPending){
            Task.current.Succeed();
        }
    }
}

```

```

}

[Task]
public void PickDeadAgent(){
    Collider[] colliders = Physics.OverlapSphere(transform.position, 130.0f);
    DeadAgent closestDeadAgent = null;
    float closestDistance = float.MaxValue;
    foreach (Collider collider in colliders){
        DeadAgent deadAgent = collider.GetComponent<DeadAgent>();
        if (deadAgent != null && deadAgent.enabled){
            float distance = Vector3.Distance(transform.position, deadAgent.transform.position);
            if (distance < closestDistance){
                closestDeadAgent = deadAgent;
                closestDistance = distance;
            }
        }
    }
    if (closestDeadAgent != null){
        Vector3 dest = new Vector3(closestDeadAgent.transform.position.x, 0,
closestDeadAgent.transform.position.z);
        agent.SetDestination(dest);
        Task.current.Succeed();
    }
}
}

```

A6 – Find Dead (.txt)

```

tree("Root")
while IsAgentAlive
    sequence
    PickDeadAgent
    MoveToDestination

```

A7 – Health Manager (.cs)

```

using System.Collections;
using UnityEngine;
using UnityEngine.SceneManagement;

public class HealthManager : MonoBehaviour{
    public float healthCount;
    public float healthLossRate;
    public float immunity;
    public float DiseaseSeverity;
    float BaseImmunity;
    public bool contaminationOccured = false;
    public bool isDead = false;
    public DeadAgent deadAgent;
    Animator anim;
    //public GameObject uiPanel;
    private int code = 0;
    private double increaseTime = 0.015;
    public double time = 0f;

    void Start(){

```

```

anim = GetComponent<Animator>();
BaselImmunity = PlayerPrefs.GetFloat("BaselImmunity");
immunity = Random.Range(BaselImmunity, 0.600f);
healthCount = Random.Range(25, 101);
DiseaseSeverity = PlayerPrefs.GetFloat("DiseaseSev");

}

void Update(){
    healthCount -= healthLossRate;
    time += increaseTime;
    anim.SetFloat("Health", healthCount);
    if (healthCount >= 70){
        healthyState();
    } else if (healthCount < 70 && healthCount >= 40){
        sickState();
    } else if (healthCount < 40 && healthCount > 0){
        contagiousState();
    } else{
        deadState();
    }
}

void healthyState(){
    healthLossRate = 0.001f;
    CheckAtHospital();
    if (!contaminationOccured){
        CheckForContamination();
    }
    GetComponent<Robot_AI>().SetHealthyState(true);
}

void sickState(){
    healthLossRate = 0.005f * DiseaseSeverity;
    CheckAtHospital();
    GetComponent<Robot_AI>().SetHealthyState(false);
}

void contagiousState(){
    healthLossRate = 0.01f * DiseaseSeverity;
    CheckAtHospital();
    GetComponent<Robot_AI>().SetHealthyState(false);
}

void deadState(){
    healthLossRate = 0.0f;
    healthCount = 0f;
    increaseTime = 0;
    transform.Rotate(90, 0, 0);
    transform.position = new Vector3(transform.position.x, 0.6f, transform.position.z);
    GetComponent<Robot_AI>().SetAliveState(false);
    deadAgent.EnableScript();
    //Movement_Test findDeadComponent = GetComponent<Movement_Test>();

```

```

    if (code == 0){
        SceneSpawner.Instance.AgentDied();
        code++;
    }
}

void CheckForContamination(){
    HealthManager[] allAgents = GameObject.FindObjectsOfType<HealthManager>();
    foreach (HealthManager agent in allAgents){
        if (agent != this && agent.healthCount < 40 && healthCount > 0 ){
            float distance = Vector3.Distance(transform.position, agent.transform.position);
            if (distance < 2.0f && Random.value < (1-immunity)){ //Distance and Probability of catching
                ReduceHealth(20.0f);
                contaminationOccured = true;
            }
        }
    }
}

void CheckAtHospital(){
    MedicalBox[] allMedicalBoxes = GameObject.FindObjectsOfType<MedicalBox>();
    foreach (MedicalBox medBox in allMedicalBoxes){
        if (healthCount < 90 && healthCount > 0 ){
            float distance = Vector3.Distance(transform.position, medBox.transform.position);
            if (distance < 4.0f){
                IncreaseHealth(2.0f);
            }
        }
    }
}

public void IncreaseHealth(float amount){
    healthCount += amount;
    if(immunity<= 0.85){
        immunity += 0.0005f;
    }
}

public void ReduceHealth(float amount){
    healthCount -= amount;
}
}

```

A8 – Genes (.cs)

using UnityEngine;

```

public class Genes : MonoBehaviour{
    Renderer myRenderer;
    HealthManager healthManager;

    void Start(){
        myRenderer = GetComponent<Renderer>();
    }
}

```

```

        myRenderer.material.SetColor("_Color", Color.white);
        healthManager = transform.parent.GetComponent<HealthManager>();
    }

    void Update(){
        float healthPercentage = healthManager.healthCount / 100f;
        Color newColor = Color.Lerp(Color.white, new Color(1f, 0.2f, 0.2f), 1f - healthPercentage);
        myRenderer.material.SetColor("_Color", newColor);
    }
}

```

A9 – Medical Box (.cs)

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.AI;

public class MedicalBox : MonoBehaviour{

}

```

A10 – Rotation (.cs)

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Rotation : MonoBehaviour{
    public float rotationSpeed = 30f;

    void Update(){
        transform.Rotate(Vector3.up * rotationSpeed * Time.deltaTime);
    }
}

```

A11 – Main Menu (.cs)

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class MainMenu : MonoBehaviour{

    public void PlayGame (){
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
    }

    public void StopGame(){
        SceneManager.LoadScene(0);
    }
}

```


A12 – Simulation Settings (.cs)

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;
using UnityEngine.UI;
using UnityEngine.SceneManagement;

public class CombinedScript : MonoBehaviour
{
    [SerializeField] private Slider spawnerSlider;
    [SerializeField] private TextMeshProUGUI spawnerSliderText;
    private int updatedNumAgents = 20;

    [SerializeField] private Slider diseaseSlider;
    [SerializeField] private TextMeshProUGUI diseaseSliderText;
    private float updatedDiseaseSeverity = 1.0f;

    [SerializeField] private Slider immunitySlider;
    [SerializeField] private TextMeshProUGUI immunitySliderText;
    private float updatedImmunity = 0.000f;

    void Start(){
        spawnerSlider.onValueChanged.AddListener((v) => {
            spawnerSliderText.text = v.ToString("0");
            updatedNumAgents = Mathf.RoundToInt(v);
            PlayerPrefs.SetInt("NumAgents", updatedNumAgents);
        });

        diseaseSlider.onValueChanged.AddListener((v) => {
            diseaseSliderText.text = v.ToString("0.0");
            updatedDiseaseSeverity = v;
            PlayerPrefs.SetFloat("DiseaseSev", updatedDiseaseSeverity);
        });

        immunitySlider.onValueChanged.AddListener((v) => {
            immunitySliderText.text = v.ToString("0.000");
            updatedImmunity = v;
            PlayerPrefs.SetFloat("BaseImmunity", updatedImmunity);
        });
    }
}
```

6.0 References

Enjuanes, L., Sola, I., Zúñiga, S., Honrubia, J.M., Bello-Pérez, M., Sanz-Bravo, A., González-Miranda, E., Hurtado, J., Requena, R., Wang, L., Muñoz, D., Sánchez, C., Esteban, A. and Ripoll, J. (2022). Nature of viruses and pandemics: Coronaviruses. *Current Research in Immunology*. doi:<https://doi.org/10.1016/j.crimmu.2022.08.003>.

Madhav, N., Oppenheim, B., Gallivan, M., Prime Mulembakani, Rubin, E. and Wolfe, N. (2019). *Pandemics: Risks, Impacts, and Mitigation*. Nih.gov, [online] 17(3). doi:<https://doi.org/10.1596/978-1-4648-05271/pt5.ch17>.

Söderlund-Venermo, M., Varma, A., Guo, D., Gladue, D.P., Poole, E., Pujol, F.H., Pappu, H., Romalde, J.L., Kramer, L., Baz, M., Venter, M., Moore, M.D., Nevels, M.M., Ezzikouri, S., Vakharia, V.N., Wilson, W.C., Malik, Y.S., Shi, Z. and Abdel-Moneim, A.S. (2022). World Society for Virology first international conference: Tackling global virus epidemics. *Virology*, [online] 566, pp.114–121. doi:<https://doi.org/10.1016/j.virol.2021.11.009>.

www.tutorialspoint.com. (n.d.). PEAS Descriptors of Task Environment. [online] Available at: <https://www.tutorialspoint.com/peas-descriptors-of-task-environment#:~:text=Conclusion>

www.unicef.org. (n.d.). How to improve primary healthcare to prepare for future pandemics | UNICEF. [online] Available at: <https://www.unicef.org/stories/how-to-improve-primary-healthcare-to-prepare-for-pandemics#:~:text=Establish%20effective%20surveillance%20and%20response%20systems&text=That%20means%20that%20if%20an>.